Lecture 1

Intelligence: the ability to process information to inform some future decision or future action

Artificial Intelligence: the practice of building algorithms

Machine Learning: a subset of artificial intelligence that focuses on not explicitly programming the computer, but just trying to learn some patterns within the data to make those decisions

Deep Learning: a subset of machine learning which focuses on doing that exact process with neural networks from raw observations

Why Deep Learning?

Traditional Machine Learning: Humans manually defined features

Deep Learning: the network automatically learns to detect low-level features from high-level features

The Perceptron: Forward Propagation

X: Input

W: Weight

g:

Activation Functions: introducing non-linearities into the network

-> allows us to approximate arbitrarily complex functions

Common activation functions: Sigmoid Function, Hyperbolic Tangent, Rectified Linear Unit (ReLU)

Building Neural Networks with Perceptrons

Single Perceptron

$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j w_j$$

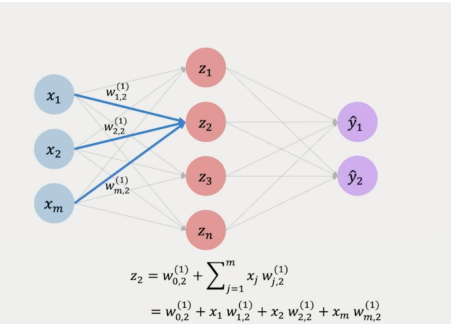
Multi Output Perceptron: Dense layer

Pytorch provide us with these dense or linear layer

Single Layer Neural Network

Hidden layer: placed between output and input layer, we don't directly observe the data which happens in this layer

We need two weight matrices W_1 (from inputs to hidden), W_2 (from hidden to outputs)



Deep Learning> a model where the final output is created as a hierarchical combination of going deeper into these linear followed by non-linear operations

Two inputs, one outputs example

Input1: how many lectures I attend

Input2: how many hours I spend on my final project

To train the model, model has to also understand when it makes bad predictions, what does a bad prediction mean >> it has to be able to quantify how bad a prediction is

From several students

We want to find neural networks that minimize our loses

Softmax cross entropy loss: measuring the difference or the distance between two probability distributions; two binary probability.

Loss Optimization: We want to find the network weights that achieve the lowest loss

Algorithm

1. Initialize weights randomly $\sim\mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
4. Update weights, $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
5. Return weights

In tensorflow we can see this represented the same way

Computing gradients: Backpropagation

Neural Networks in Practice: Optimization

Optimization through gradient descent

$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$$

η: learning rate

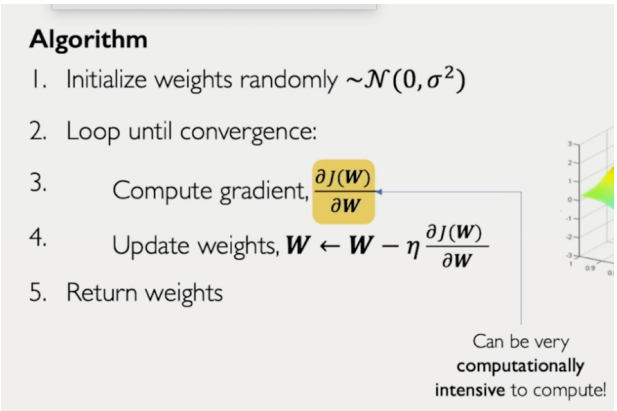
Adaptive Learning Rates > no more fixed learning rates

Gradient Descent Algorithm

Adam: extremely well used

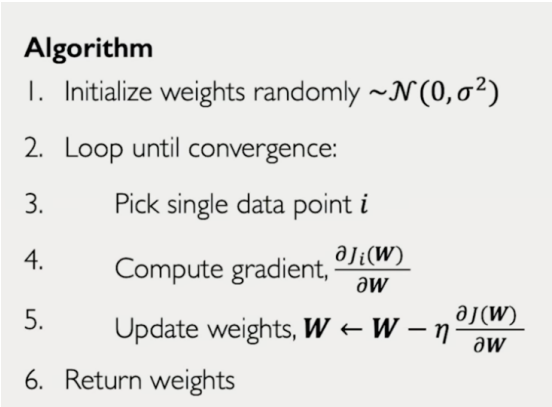
Gradient Descent

We compute every point...



Stochastic Gradient Descent

Compute just one data point, not my entire dataset, and I'll get result quickly



Mini-batches while training: More accurate estimation of gradient; allows for lager learning rates

>> Mini-batches lead to fast training

Overfitting: too complex, extra parameters, does not generalize well

If I have a very small dataset but a very large network, I'll learn a model that just memorizes all of the data in my dataset and it' doing something bad because it has the

power to memorize everything in the training set and don't see test set, so all they can see is my training set what I've gave to them, and they'll do what it's supposed to do and learn exactly the training set to the fully capacity, but then when I show it more test data, it's not going to be very trustful to the training set.

Regularization: a technique that constrains our optimization problem to discourage complex models

- we need to improve generalization of our model on unseen data

1. Dropout: during training, we're going to randomly set some activations to 0, which forces to network to not rely on any I node

2. Early Stopping: we monitor the deviation between our training loss and our test loss, how well the model is doing. At some point, the test loss starts to memorize data in the training loss, which results in the test loss to go up a little bit, and this pattern continues for the rest of training

>> so we take the model where before this point